

Hi3519DV500 Linux wms_server CPU 198% 异常占用修复指南

一、问题现象

Hi3519DV500 开发板烧录系统后，通过串口执行 `top` 命令查看系统资源占用，发现 `wms_server` 进程持续占用约 **198% CPU**（双核全满），严重消耗系统资源：

```
Tasks: 83 total,  2 running,  81 sleeping,  0 stopped,  0 zombie
Mem:   103740K total,  89579520 used,  16650240 free,   573440 buffers
200%cpu 200%user   0%nice   0%sys   0%idle   0%iow   0%irq   0%sirq   0%host
  PID USER      PR  NI  VIRT  RES  SHR S[%CPU] %MEM    TIME+  ARGS
 1291 10         20   0   41M  3.7M  3.2M R   198    3.7    3:57.25 wms_server
```

系统基本情况：

- 产品： `ipcamera_hispark_hi3519dv500_linux`（IP Camera 场景）
- 内存：仅 103MB 可用（`mem=256M` bootargs）
- CPU：双核 ARM64
- 影响：`wms_server` 吃掉所有 CPU 时间片，其他进程调度受阻，系统整体响应迟缓

二、根因分析

2.1 背景：display_layer_stub 的由来

在此之前，为解决 `mem=256M` 下的 display 刷屏问题（详见 `addThirdDocs/hi3519dv500_linux_mem256m_display_disable_guide.md`），已将 SoC display HAL 替换为 stub 实现：

文件： `device/soc/hisilicon/common/hal/display/display_layer_stub.c`

该 stub 的设计原则是"所有函数返回 `DISPLAY_SUCCESS` 的空操作"，使得上层 graphic 子系统不会因 display HAL 失败而崩溃。`StubGetLayerBuffer` 还分配了 `1920x1080x4 = 8MB` 虚拟缓冲区，确保上层指针访问不会段错误。

2.2 CPU 198% 的触发链路

问题出在 `wms_server` 的主循环与 stub 的 `waitForVBlank` 配合上：

```
wms_server 启动
→ main() (wms.cpp:31)
→ HiFbdevInit() // stub 全部返回 SUCCESS, "成功"初始化
→ layerData_ -> virAddr 非空 // stub 分配了虚拟 buffer
→ 进入主循环

主循环 (wms.cpp:42-46):
while (1) {
    MainTaskHandler(); // layerData_ 非空 → 不 early return, 执行渲染逻辑
    WaitForVBlank(); // ← 关键: stub 立即返回 SUCCESS, 无任何阻塞!
}
// 循环零延迟 → 双核各 ~99% CPU = 198%
```

2.3 三个条件同时满足才触发

条件	来源	说明
① WaitForVBlank 不阻塞	display_layer_stub.c:StubWaitForVBlank	原实现直接返回 SUCCESS, 无 sleep/nanosleep
② layerData_ 非空	display_layer_stub.c:StubGetLayerBuffer	分配了虚拟缓冲区, MainTaskHandler 不提前 return
③ wms_server 被启动	init_linux_3519dv500_openharmony_release.cfg	init 阶段执行 start wms_server

三个条件缺一不可。在正常有显示器的平台上，WaitForVBlank 会阻塞等待硬件 VBlank 信号（约 16.67ms@60fps），循环不会空转。但在 stub 环境下，条件 ① 被破坏。

2.4 关键代码文件

文件路径	作用	关键行
foundation/window/window_manager_lite/services/wms/wms.cpp	wms_server 主入口	42-46 (主循环)
foundation/window/window_manager_lite/services/wms/lite_wm.cpp	窗口管理核心逻辑	MainTaskHandler()
foundation/window/window_manager_lite/services/wms/lite_vblank.h	VBlank 管理	WaitForVBlank() → OHOS::WaitForVBlank()
foundation/graphic/graphic_utils_lite/frameworks/hals/hi_fbdev.cpp	显示 HAL 封装	WaitForVBlank() 调用 layerFuncs->WaitForVBlank
device/soc/hisilicon/common/hal/display/display_layer_stub.c	display stub 实现	StubWaitForVBlank — 根因所在
vendor/.../init_linux_3519dv500_openharmony_release.cfg	init 启动配置	start wms_server 启动命令 + 服务定义

2.5 代码调用时序图

```
init (PID 1)
└─ start wms_server
    └─ wms_server (main, wms.cpp:31)
        └─ HiFbdevInit() // stub: 全部 SUCCESS
            └─ DisplayInit() → LayerInitialize() // 注册 stub 函数指针
            └─ OpenLayer() → GetDisplayInfo() // 返回 1920x1080
            └─ CreateLayer() // SUCCESS
```

```
|   |↳ SetLayerVisible(true)                // SUCCESS
|   |↳ AllocDisplayBuffer()                // 分配 8MB 虚拟 buffer
|
|↳ LiteVBlank::Run()                      // 启动 vBlank 通知线程
|
|↳ while(1) {                             // ← 主循环（无 sleep!）
|   |↳ MainTaskHandler()
|   |   |↳ ProcessUpdates()                // 无窗口，几乎无操作
|   |   |
|   |   |↳ WaitForVBlank()
|   |       |↳ hi_fbdev.cpp:WaitForVBlank()
|   |           |↳ layerFuncs->WaitForVBlank()
|   |               |↳ stubWaitForVBlank()
|   |                   |↳ return DISPLAY_SUCCESS; // 立即返回！
|   }
}
```

三、修复方案

采用**双重修复策略**：

- 1. **方案一（根治）**：从 init 配置中移除 wms_server 启动项 — IP Camera 无显示器，不需要窗口管理器
- 2. **方案二（安全网）**：在 stub 的 WaitForVBlank 中添加 nanosleep — 防止将来重新启用 wms_server 时再次出现忙循环

3.1 方案一：禁用 wms_server 启动

涉及文件

文件	操作
vendor/hisilicon/hispark_hi3519dv500_linux/init_configs/init_linux_3519dv500_openharmony_release.cfg	移除 wms_server
vendor/hisilicon/hispark_hi3519dv500_linux/init_configs/init_linux_3519dv500_openharmony_debug.cfg	移除 wms_server

操作一：移除启动命令

在 "init" job 的 `cmds` 列表中，删除 `"start wms_server"` 这一行：

修改前：

```
"start media_server",
"start wms_server",
"start hiview",
```

修改后：

```
"start media_server",
"start hiview",
```

两个 `cfg` 文件（`release` 和 `debug`）都需要做相同修改。

操作二：移除服务定义

在 "services" 数组中，删除整个 wms_server 服务定义块：

修改前：

```
    }, {
      "name" : "wms_server",
      "path" : ["/bin/wms_server"],
      "uid" : 10,
      "gid" : 10,
      "once" : 1,
      "importance" : 0,
      "caps" : [1,23]
    }, {
      "name" : "bundle_daemon",
```

修改后：

```
    }, {
      "name" : "bundle_daemon",
```

两个 cfg 文件（release 和 debug）都需要做相同修改。

注意：consistency_src.json 中也有 wms_server 的记录，该文件是构建一致性校验用的，不影响运行时行为。如果需要彻底清理可以从该文件中也移除，但非必需。

3.2 方案二：修复 stub WaitForVBlank

涉及文件

文件	操作
device/soc/hisilicon/common/hal/display/display_layer_stub.c	添加 nanosleep

修改内容

修改前：

```

#include <stdlib.h>
#include <string.h>
#include "display_layer.h"

/* ... 其他函数 ... */

static int32_t StubWaitForVBlank(uint32_t devId, uint32_t layerId, int32_t timeOut)
{
    (void)devId;
    (void)layerId;
    (void)timeOut;
    return DISPLAY_SUCCESS;
}

```

修改后：

```

#include <stdlib.h>
#include <string.h>
#include <time.h>          /* 新增: nanosleep */
#include "display_layer.h"

/* ... 其他函数不变 ... */

static int32_t StubWaitForVBlank(uint32_t devId, uint32_t layerId, int32_t timeOut)
{
    (void)devId;
    (void)layerId;
    (void)timeOut;
    struct timespec ts = {0, 16666000}; /* 16.666ms ≈ 60fps */
    nanosleep(&ts, NULL);
    return DISPLAY_SUCCESS;
}

```

关键细节：

1. 使用 `nanosleep` 而非 `usleep`：该文件以 `-std=c99` 编译且 `-werror`，`usleep` 属于 POSIX 扩展而非 C99 标准，会导致隐式声明错误。`nanosleep` 在 `<time.h>` 中声明，兼容性更好。
2. 延时 16.666ms：对应 60fps 的 VBlank 间隔，与正常显示器帧率一致。这个值不会影响系统性能（因为方案一已禁用 `wms_server`），仅作为安全网存在。

四、编译验证

4.1 编译命令

```

# 在 Docker 容器内或通过 docker exec 执行
docker exec phs-openphs-dev bash -lc 'cd /workspace/projects/phs/openphs/HiYearningOH4/ohos
&& \
    hb build -T wms_server 2>&1 | tail -20'

```

预期输出：

```
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:01:xx
```

4.2 更新 init 配置到 rootfs

`hb build -T wms_server` 只编译二进制，不会更新 rootfs 中的 init.cfg。需要额外编译 init_configs target：

```
docker exec phs-openphs-dev bash -lc 'cd /workspace/projects/phs/openphs/HiYearningOH4/ohos
&& \
  hb build //vendor/hisilicon/hispark_hi3519dv500_linux/init_configs:init_configs 2>&1 |
tail -10'
```

提示： 如果做全量编译 (`rm -rf out && hb build -f`)，以上两步会自动包含，无需单独执行。

4.3 验证方法

验证 1：确认 rootfs init.cfg 中无 wms_server

```
docker exec phs-openphs-dev bash -c 'grep -c "wms_server"
/workspace/projects/phs/openphs/HiYearningOH4/ohos/out/hispark_hi3519dv500/ipcamera_hispark_
hi3519dv500_linux/rootfs/etc/init.cfg'
```

预期输出：0（无匹配）

也可以更直观地查看所有服务和启动命令：

```
docker exec phs-openphs-dev bash -c 'cat
/workspace/projects/phs/openphs/HiYearningOH4/ohos/out/hispark_hi3519dv500/ipcamera_hispark_
hi3519dv500_linux/rootfs/etc/init.cfg | python3 -c "import sys,json; d=json.load(sys.stdin);
svcs=[s["name"] for s in d["services"]]; print("Services:", svcs); cmds=[c for j in
d["jobs"] for c in j["cmds"] if c.startswith("start ")]; print("start cmds:",
cmds)''
```

预期输出中 `services` 和 `start cmds` 列表都不包含 `wms_server`。

验证 2：确认 stub libdisplay_layer.so 包含 nanosleep 引用

```
docker exec phs-openphs-dev bash -c 'readelf -s
/workspace/projects/phs/openphs/HiYearningOH4/ohos/out/hispark_hi3519dv500/ipcamera_hispark_
hi3519dv500_linux/rootfs/usr/lib/libdisplay_layer.so | grep nanosleep'
```

预期输出：

```
10: 0000000000000000      0 FUNC      GLOBAL DEFAULT  UND nanosleep
```

验证 3：烧录后 top 验证

烧录新固件后，通过串口执行 `top`，确认：

- 进程列表中无 `wms_server`（方案一生效）

- CPU idle > 0% (不再是 200% user)
- 可用内存增加约 3.7MB (wms_server 不再运行)

五、烧录后验证结果（预期）

修复前：

```
200%cpu 200%user  0%nice  0%sys  0%idle ...
1291 10      20    0  41M 3.7M 3.2M R 198   3.7   3:57.25 wms_server
```

修复后：

```
~0%cpu  ~0%user  0%nice  0%sys  ~99%idle ...
(wms_server 不在进程列表中)
```

六、恢复方法

如果将来需要重新启用 display 和 wms_server（例如连接了 HDMI 显示器），按以下步骤恢复：

6.1 恢复 wms_server 启动

在 `init_linux_3519dv500_openharmony_release.cfg`（和 debug 版本）中：

1. 在 "init" → "cmds" 的 "start media_server" 后面添加：

```
"start wms_server",
```

2. 在 "services" 数组的 media_server 服务块后面添加：

```
}, {
  "name" : "wms_server",
  "path" : ["/bin/wms_server"],
  "uid" : 10,
  "gid" : 10,
  "once" : 1,
  "importance" : 0,
  "caps" : [1,23]
```

6.2 恢复 display HAL（如果需要真实显示输出）

```
# 在 Docker 容器内执行
cd /workspace/projects/phs/openphs/HiYearningOH4/ohos

# 恢复 BUILD.gn（还原 SoC display_layer 构建）
cp device/soc/hisilicon/common/hal/display/BUILD.gn.bak \
  device/soc/hisilicon/common/hal/display/BUILD.gn

# 删除 stub 文件
rm device/soc/hisilicon/common/hal/display/display_layer_stub.c
```

```
# 恢复 init 配置
cp device/board/hisilicon/hispark_hi3519dv500/linux/system/cfg/init.Hi3516DV300.cfg.bak \
  device/board/hisilicon/hispark_hi3519dv500/linux/system/cfg/init.Hi3516DV300.cfg

# 恢复 HCS 配置
cp vendor/hisilicon/hispark_hi3519dv500_linux/hdf_config/device_info/device_info.hcs.bak \
  vendor/hisilicon/hispark_hi3519dv500_linux/hdf_config/device_info/device_info.hcs

# 全量重新编译
rm -rf out && hb build -f
```

重要：如果仅恢复 wms_server 但继续使用 stub（不恢复真实 display HAL），stub 中的 nanosleep 安全网会确保 wms_server 不会忙循环，CPU 占用约 0%。但 wms_server 仍然分配了 8MB 虚拟 buffer，在 103MB 可用内存的系统上是不必要的浪费。

七、修改文件清单

文件	修改类型	说明
device/soc/hisilicon/common/hal/display/display_layer_stub.c	修改	StubWaitForVBlank 添加 nanosleep(16.6ms)，添加 <code>#include <time.h></code>
vendor/.../init_linux_3519dv500_openharmony_release.cfg	修改	移除 start wms_server 和 wms_server 服务定义
vendor/.../init_linux_3519dv500_openharmony_debug.cfg	修改	移除 start wms_server 和 wms_server 服务定义

八、经验总结

- Stub 设计原则：**禁用硬件功能的 stub 不仅要让函数返回成功，还必须**模拟原函数的时序行为**。
`waitForVBlank` 的核心语义是"阻塞等待下一帧"，stub 应该用 `nanosleep` 模拟这个延迟，而不是简单地立即返回。
- 无头系统的服务裁剪：**IP Camera 等无显示器的小型系统，应从 init 配置中移除不需要的 GUI 相关服务（`wms_server`、`media_server` 等），既节省 CPU 也节省内存。
- init 配置的编译传递：**修改 `vendor/.../init_configs/` 下的 `cfg` 文件后，需要编译对应的 `init_configs` target 才能更新到 rootfs 的 `etc/init.cfg`。单独编译某个 binary target（如 `hb build -T wms_server`）不会触发 init 配置的更新。
- C99 编译环境下的注意事项：**本项目使用 `-std=c99 -werror` 编译，`usleep` 等 POSIX 函数会导致隐式声明错误。应使用 `nanosleep`（`<time.h>`）替代，它在 C99 模式下可用。

修复日期： 2026-05-19
平台： Hi3519DV500 (ipcamera_hispark_hi3519dv500_linux)
OpenHarmony 版本： 4.0-Release
关联文档： `addThirdDocs/hi3519dv500_linux_mem256m_display_disable_guide.md` (display 刷屏问题修复)